

BONUS

7. Considerare il problema *DIFF-SAT* che prende in input un'istanza Φ di *SAT* e restituisce vero se e solo se la formula Φ ammette un'assegnazione di verità alle variabili avente l'ulteriore proprietà che ogni clausola formata da almeno due letterali contiene sia un letterale vero che un letterale falso. Mostrare che *X* è NP-Completo. (suggerimento: riduzione da *3-SAT*).

Dobbiamo mostrare che *3-SAT* $\leq_p$ *DIFF-SAT* e che *DIFF-SAT* è in NP.

Iniziamo mostrando che *DIFF-SAT* ha un certificatore polinomiale: data un'istanza ϕ del problema ed una sua assegnazione di verità, il certificatore non dovrà fare altro che leggere ogni singola clausola: se le variabili all'interno di una clausola sono tutte false oppure tutte vere, restituisce NO; se per ogni clausola c'è almeno una variabile vera ed una falsa, restituisci YES.

L'algoritmo è già lineare, ma potremmo migliorarlo ulteriormente nei casi medi facendo sì che passi alla clausola successiva non dopo averla letta per intero ma non appena incontra le due variabili richieste.

Per definizione di trasformazione polinomiale, sappiamo che $X \leq_p Y$ se dato un qualsiasi input x in X , possiamo costruire un input y tale che x è istanza YES di X se e solo se y è istanza YES per Y . Per verificare la riduzione, dobbiamo quindi mettere in relazione istanze arbitrarie di *3-SAT* con istanze arbitrarie di *DIFF-SAT* che andremo a costruire.

Data ϕ istanza di *3-SAT*, sappiamo che avrà la seguente forma: $C_1 \text{ AND } C_2 \text{ AND } \dots \text{ AND } C_n$. Costruiamo ϕ' istanza di *DIFF-SAT* aggiungendo una variabile x ad ogni clausola C_i originale ed una x negata dopo l'ultima clausola C_n . Avremo che $\phi' = (C_1 \text{ OR } x) \text{ AND } (C_2 \text{ OR } x) \text{ AND } \dots \text{ AND } (C_n \text{ OR } x) \text{ AND not } x$.

- Se l'espressione originaria ϕ è soddisfacibile, vuol dire che all'interno di ogni sua clausola c'era almeno una variabile posta a true: porremo quindi x a false per garantire che ogni clausola di ϕ' abbia almeno un letterale false, e di conseguenza l'ultima variabile not x sarà vera e la formula ϕ' risulterà anch'essa soddisfacibile.
- Se l'espressione originaria ϕ non è soddisfacibile, vuol dire che c'è almeno una sua clausola in cui ogni variabile è posta a false: porremo quindi x a true per garantire che ogni clausola di ϕ' abbia almeno un letterale true, e di conseguenza l'ultima variabile not x risulterà false e la formula ϕ' risulterà anch'essa non soddisfacibile. È anche possibile che in questo caso vi sia stata una clausola di ϕ con tutte le variabili poste a true, dunque la corrispondente clausola in ϕ' non avrà variabili poste a false: questa espressione ha comunque risposta NO come dovrebbe, in quanto il not x finale è false.

Provare che $\text{Atm} \leq_m L$ con $L = \{ \langle M \rangle \mid M \text{ è una MdT e } |L(M)| \leq 4 \}$

(Opzionale) Sia T_5 una MdT tale che $|L(T_5)| = 5$: chiaramente T_5 non appartiene ad L .

Sia T una MdT tale che $\langle T \rangle \in L$.

Progettiamo una funzione $f(\langle M, w \rangle) = \langle S \rangle$, dove S , su input x , simula M su input w :

- Se M si ferma e rifiuta, allora S rifiuta x ;
- Se M accetta, allora S simula T su input x : se T accetta, allora x , allora S accetta, altrimenti rifiuta.

Abbiamo costruito una funzione $f(\langle M, w \rangle) = \langle M \rangle$ che è calcolabile, poiché $\langle M, w \rangle \in \text{Atm} \Leftrightarrow f(\langle M, w \rangle) \in L$

7. Una palestra organizza una trasferta. Gli atleti sono suddivisi in n discipline sportive e devono viaggiare su due bus A e B. Gli atleti che praticano una stessa disciplina non possono viaggiare su bus diversi. Sul bus A possono viaggiare esattamente a atleti (su B possono viaggiare esattamente gli altri). Si consideri il problema di decisione:
SPORT: È possibile selezionare k discipline in modo tale da riempire il bus A?
Provare che BUS è NP-completo. (Suggerimento: usare *SUBSET-SUM*).

- Un insieme di n discipline sportive: $D = \{d_1, d_2, \dots, d_n\}$.
- Ogni disciplina d_i ha un certo numero di atleti $a_i \in \mathbb{N}$.
- Un numero intero w : capacità esatta del bus A.
- Un intero k : numero di discipline da scegliere.

Costruzione della riduzione

Data un'istanza di *SUBSET-SUM* ($S = \{s_1, \dots, s_n\}, t$), costruiamo un'istanza di *SPORT*:

- Per ogni $s_i \in S$, crea una disciplina d_i con $a_i = s_i$ atleti.
- Imposta $w = t$ (il valore da raggiungere con la somma).
- Imposta k libero, ma lo poniamo uguale a n , così da non porre vincoli extra.

Allora la domanda diventa: esiste un sottoinsieme di discipline con somma esattamente w ? (ovvero: un sottoinsieme degli s_i che somma t : esattamente *SUBSET-SUM*)

4. Correttezza della riduzione

- Se esiste una soluzione a *SUBSET-SUM*, esiste una selezione di discipline con somma $w = t$.
- Se esiste una selezione di discipline con somma esattamente w , allora c'è un sottoinsieme degli s_i che somma t .

Quindi, la riduzione è corretta e polinomiale.

Conclusione

- *SPORT* è NP (verificabile in tempo polinomiale)
- *SPORT* è NP-difficile (riduzione da *SUBSET-SUM*)
- Quindi, *SPORT* è NP-completo.

(16 punti) Date due espressioni regolari E_1 ed E_2 , la notazione $E_1 = E_2$ indica che esse denotano lo stesso linguaggio. Per ognuna delle seguenti relazioni, dire se essa è vera o falsa illustrando il linguaggio comune a E_1 e E_2 oppure fornendo due stringhe che mostrano che $E_1 \neq E_2$; risposte non giustificate non saranno valutate.

- (a) $\emptyset^* = \epsilon^*$ (b) $(0 \cup 1)^* = 0^* \cup 1^*$ (c) $(0 \cup 1)(1^* \cup 0) = 00 \cup 11^* \cup 10 \cup 11^*$
(d) $(01)^* = 0^*1^*$ (e) $(000^* \cup 111^*) = (00 \cup 11)^*$ (f) $(0^* \cup 1^*)1^* = (01)^* \cup 1^*$.

Vero.

- $\emptyset^* = \{\epsilon\}$ perché lo star include sempre la concatenazione zero volte, cioè ϵ ; non esistono concatenazioni non-vuote di elementi di $\emptyset$.
- $\epsilon^* = \{\epsilon\}$ perché concatenando qualunque numero di ϵ si ottiene sempre ϵ .

Quindi il linguaggio comune è $\{\epsilon\}$ (e.g., ϵ appartiene a entrambi; nessun'altra stringa appartiene).

B: falsa 01 solo sx

C: falsa 01 solo sx (11* due volte -> typo?)

D: falsa 001 solo dx

E: falsa 000 solo sx (con o senza typo)

F: falsa 0101 solo dx